# Typographics Art: everything in one file This file contains the README and the main documents of the open-source project Typographics Art (https://typographics-art.org, https://github.com/typographics-art/typogram). It is meant to be handed to an AI assistant as a whole. Good first questions: What is Typographics Art? How
Can a machine still read a work? How big do the PDFs get, and why? --- <!-- README.md --> ## Overview Typographics Art Four works: Nikola Tesla, The Great Wave, The Wanderer above the Sea of Fog and Frederick Douglass, each set from a whole text; the original image is the small inset Stationery for the AI age. A docume
glance, and a machine reads exactly as shown. Try it on typographics-art.org · Gallery · Definition · AI readability · Text fidelity · AI benefit (measured) · Prior art · Deutsch --- ## Ask your AI what this is The README and the main documents in one file: **[about/typographics-art.md](https://typographics-art.org/about/typographic
characters, also at typographics-art.org/llms-full.txt). Or hand it the same text as a one-pager, made with the technique itself: **[about/typographics-art.pdf](https://typographics-art.org/about/typographics-art.pdf)**. Then ask: *What is Typographics Art, and can you still read this
centuries, letterhead, a signature and a seal showed at a glance what a paper was and who it was from. **Typographics Art** brings that to the AI age, in three layers: 1. **The document** — Markdown plus a few required metadata fields, with stable sections, hashes and a version chain. On its own, already a useful "llms.txt wi
provenance** — keys on domains, signatures, a visible seal, a verify tool. This is the piece missing in the world today: signatures exist for software and for photos, but not yet a widespread, readable one for texts addressed to AIs. Keys and signatures **can** also be anchored in a public transparency log or chain — the format al
contract are defined (OpenTimestamps on Bitcoin, Sigstore Rekor, a chain such as DigiByte, not tied to one), open to the community. (Shipping in this release: keys, signatures, the seal and the verify tool. Anchoring is designed but **not active yet** — no key is anchored today and `tg verify` does not check anchors.) 3. **The pictu
core, but the reason people understand the format without reading the spec: a seal on an envelope needs no explanation. It turns a crypto protocol into an object you recognise, show and hang up. On top of that, traffic forms: bundles that collect several works, sealed sends, handoff packages between people and their agents, an
coding agents. **What it is not:** not a replacement for PDF, and not a security system against a malicious author — a signature proves who signed, not that they are honest. It is a possibility, not a guarantee: whether an agent actually runs the check depends on verify tools landing where models run, and on issuers whose seal peop
plainly: stationery for the AI age — letterhead, signature, seal and envelope, things people have used for centuries to show origin and integrity, translated into a format a machine can check and a human still understands at once. Smaller than "everything in one file", but true, and it did not exist before. ### How the picture works **ASCII a
picture out of characters, but the characters mean nothing. A person sees a face; a machine sees `@@%#*+=-:.`. The text is gone. **Typographics Art** (TG Art) builds a picture out of a *real text*. Every character of the text is set once, in its original order. Only its typography changes: size, weight and colour per glyph (width is an option
person sees the picture. A language model, a search engine or a screen reader reads the text, exactly as it was written. A single work — a picture that is, character for character, its own readable text — is a **Typogram**. A Typogram is delivered as ordinary HTML, SVG or PDF (the text stays real, so any program reads it back), and it ca
structured data or sealed media inside it. "Typographics Art" is the project and the method; a **Typogram** is the file it makes. | | ASCII art | Typographics Art | |---|---|---| | What is chosen | the **character** for each cell of a grid | the **typography** for each character of a text | | The text | lost, replaced by shading characters | every character k
reading order (whitespace collapsed) | | What a machine reads | `@@%%##**++==--::..` | *"Wer reitet so spät durch Nacht und Wind? ..."* | | Fonts | monospace, one size | a variable font: size, weight and colour per glyph (width is an optional axis) | | Format | plain text | HTML, SVG, PDF with a text layer | ### What a person sees, and what an AI re
The portrait of Frederick Douglass above is set from his own *Narrative* (1845), chapters V–VIII. Open [`gallery/douglass.svg`](https://typographics-art.org/gallery/douglass.svg), select all and copy, or hand the file to any model: ```text As to my own treatment while I lived on Colonel Lloyd's plantation, it was very similar to that of the other sla
children. I was not old enough to work in the field, and there being little else than field work to do, I had a great deal of leisure time. ... ``` That is the whole point: **the artwork is also the document.** The text is the content; what the picture *shows* still gets a short description of its own, which the generator writes into the file (the SVG title, the HTML
figure label). ## Readable for AI, measured We do not just claim it. Every number below is measured and reproducible ([method](https://typographics-art.org/docs/ai-readability.md)). | How a machine gets the text | Result | |---|---| | **HTML or SVG**: reading the characters (DOM, XML) | **100 % exact.** Checked by the test suite on every commit. | |
PDF text layer in stored order (`pdftotext -raw`) | **100 % exact** in all 6 tested settings (Sonnet 18 plus Lorem ipsum, one synthetic image) | | **PDF**, tools that rebuild the visual layout (`pdftotext` default) | exact at legibility *high*; 97–99 % of the word order kept at *medium* and *free* | | **Only the picture**: OCR (Tesseract 5.5) on a PNG at 2× |
legibility *high*: **98.9–99.6 %** of words · *medium*: 88.7–92.7 % · *free*: 68.8–82.3 % | | **Only the picture**: a vision-language model (Claude) reading crops | **100 %** of 217 words, at *high*, *medium* and *free* (first sample) | So: **give machines the HTML or SVG**, and they get the text perfectly, at any artistic setting. PDF works the same when
read in stored order, and with every tool at legibility *high*. If only an image travels (a screenshot, a social post), choose legibility *high* and the text survives even pure image recognition. A first sample with a vision-language model read every word, even at *free*, where classic OCR stumbles. More models and larger samples: [help is
welcome](https://github.com/typographics-art/typogram/blob/main/CONTRIBUTING.md). One caveat for PDF: Poppler's `pdftotext` keeps only the first 50 000 small characters per page. For pages with more text than that (a one-pager of a long document), read the PDF with MuPDF, or run `pdftotext -r 600`
([details](https://typographics-art.org/docs/ai-readability.md#which-pdf-extractors-read-a-one-pager-completely)). ## One-pagers: a whole document on one page `cli/onepage.mjs` puts the complete text of a multi-page PDF (or any text file) on a single page. The letters are tiny; together they form the document's title in large type, or, with `--motif`, a
picture above the title. A person sees at a glance what the page is. A model reads every word from the PDF's text layer: the type size does not matter to it. | | | | |---|---|---| | Swiss Federal Constitution on one page **Swiss Federal Constitution** 102 pages, 228 576 characters | EU AI Act on one page **EU AI Act** 144 pages, 595 706 characters | Code review
agent instructions on one page **AI instructions** a code review agent's rules | Every example is read back with MuPDF and compared with its source; every visible character matches. 31 examples (laws, declarations, PEP 8, AI agent instructions, this repository's [AGENTS.md](https://github.com/typographics-art/typogram/blob/main/AGENTS.md))
are in [examples/](https://github.com/typographics-art/typogram/tree/main/examples), each with its published source text. ```sh node cli/onepage.mjs document.pdf --title "Short title" -o onepage.pdf --png preview.jpg node cli/onepage.mjs document.pdf --title "Short title" --motif photo.jpg -o onepage.pdf ``` **File size.** Every glyph can carry its
own weight, size and colour, and a PDF embeds a font for every weight × width × size it uses. Measured and optimised in [docs/file-size.md](https://typographics-art.org/docs/file-size.md): outlines rounded, empty accessibility tags left out, one type size by default (which also makes the picture sharper). Of the five examples with a PDF source (Swiss
constitution, German Basic Law, US Constitution, EU AI Act, Universal Declaration of Human Rights), three are *smaller* than their source (US Constitution 1.6 MB from 3.8 MB, EU AI Act 2.2 MB from 2.5 MB, Declaration 137 KB from 191 KB); the Swiss constitution is 1.6 times and the Basic Law 2.8 times larger; short plain-text sources grow most (3 KB of
instructions -> 67–105 KB). `--weights 2` makes a one-pager smaller still. The [examples table](https://github.com/typographics-art/typogram/blob/main/examples/README.md#file-size-honestly) lists every size; the original motif images are in [examples/motifs/](https://typographics-art.org/examples/motifs/). Share the PDF, not a picture of it: at this
size, a plain image is grey texture to OCR and vision models. And keep it honest: the visible title must name the document, and tiny text must never hide instructions from the people who see the page. ## The three views: picture, source, and a hash that binds them A Typogram is one file with **three guaranteed views**: - the **image layer** — the
artwork, a person's view: the text set character for character so it forms the picture; - the **data layer** — a machine's view: the exact source (Markdown, with its headings, tables, lists and code) and a schema.org JSON-LD record, carried losslessly *inside the same file* (an SVG `<metadata>` before the artwork, a `<script type="text/markdown">` +
JSON-LD in an HTML `<head>`, attached files in a PDF); - an **identity hash** — the SHA-256 of the canonical text ([docs/canonical-text.md](https://typographics-art.org/docs/canonical-text.md)), stored in the data layer, that proves both views carry the same text. What a machine reads is the **data layer**, read directly, not OCR'd from the pixels:
```sh tg extract about/typographics-art.svg --markdown # the exact source, byte for byte tg extract about/typographics-art.svg --json # the JSON-LD record (title, author, hash, ...) tg verify about/typographics-art.svg # prove the picture, the source and the hash are one text ``` `tg verify` recomputes the hash from the glyphs and requires it to equal
both the stored hash and the hash of the embedded source — so a model can trust that the readable source and the picture say the same thing, and refuse the file if they do not. The same works on the `.html` and `.pdf` views. A standalone reference, [tools/tg_extract.py](https://github.com/typographics-art/typogram/blob/main/tools/tg_extract.py)
(Python standard library only), reads and verifies the same files and produces the same hash. The format is [docs/typogram-file.md](https://typographics-art.org/docs/typogram-file.md); why a model should prefer the data layer, and the prompt-injection answer, is
[docs/ai-readability.md](https://typographics-art.org/docs/ai-readability.md#what-a-model-reads). ## Signing & verifying A Typogram can be **signed** by its issuer, so a reader — a person or an AI — checks **who published it** and that **not one character changed**. The signature covers the identity hash above; it survives copy, paste and export
and lives in the data layer, so the picture is untouched. On the website, a signed work carries a small **green yes corner**. `tg verify` gives one of three answers: | Result | Exit | Means | |---|---|---| | **VERIFIED** | 0 | signed by this issuer; picture, embedded source and signature all carry one text | | **UNSIGNED** | 2 | consistent (picture = source = hash),
but not signed | | **TAMPERED** | 1 | the signature, the hash or the two layers disagree | To check a signature you need the issuer's **public key**. Every verified issuer key is published four ways, so any verifier can reach one — including an agent whose allowlist is only npm, PyPI or GitHub: ```js // npm — @typographics-art/keys, immutable per
published version import { getIssuer, issuers } from '@typographics-art/keys'; issuers(); // ['typographics-art.org', ...] getIssuer('typographics-art.org'); // { issuer, verifiedAt, keys: [...] } ``` ```python # PyPI — tgart-keys, immutable per published version import tgart_keys tgart_keys.get_issuer('typographics-art.org') ``` - **GitHub (raw):**
`https://raw.githubusercontent.com/typographics-art/typogram/main/keys/registry.json` - **The issuer's own domain:** `https://<issuer>/.well-known/tgart-keys.json` Then verify a work against the registry — it picks the issuer the work names: ```sh tg verify work.tg.svg --keys keys/registry.json # VERIFIED / UNSIGNED / TAMPERED ``` No install
needed: the browser verifier at typographics-art.org/v does the same (nothing is uploaded), and an AI agent can call the read-only **MCP server** — `tg_verify`, `tg_extract`, `tg_keys` ([docs/mcp.md](https://typographics-art.org/docs/mcp.md)). The signature format and reproducible test vectors are in
[docs/verification.md](https://typographics-art.org/docs/verification.md); the verified-key registry and how to get a key listed in [docs/key-registry.md](https://typographics-art.org/docs/key-registry.md); issuer key publication, rotation and revocation in [docs/issuers.md](https://typographics-art.org/docs/issuers.md). Signing extends to
collections and encryption: sign a whole folder as a **bundle** with a signed index ([docs/bundles.md](https://typographics-art.org/docs/bundles.md)), **seal** a work to named recipients so only a key holder reads it ([docs/sealed.md](https://typographics-art.org/docs/sealed.md)), or hand a set of files to another party as one sealed, signed
handoff package ([docs/handoff.md](https://typographics-art.org/docs/handoff.md)). ## Hidden messages A work can carry a message that only the holder of a key can read. The work itself does not change: it holds only its public text, nothing invisible. The key records, for every character of the message, one position in that text where the same
character stands, chosen at random among all its occurrences (a book cipher). With the key and the right work, the message comes back; with any other work, the key's text fingerprint does not match. ```sh node cli/tg-secret.mjs hide work.pdf --secret "Meet at the harbour, 6 pm." # prints the key: tg1.... node cli/tg-secret.mjs reveal work.pdf --key tg1.... # prints the message ``` The examples work with an
text that contains these characters; the generator suggests sample texts when one is missing. Works for PDF (read with MuPDF), SVG, HTML and text; the web generator has the same under *Hidden message*, with 28 sample texts to choose from and suggestions for which text can carry your message. Details: [docs/hidden-messages.md](https://typographics-art.org/docs/hidden-messages.md). Keep the key like a password: with key and work, anyone
can read the message; without the key, the work reveals nothing (what a key does and does not protect: [docs/hidden-messages.md](https://typographics-art.org/docs/hidden-messages.md#what-it-protects-and-what-it-does-not)). Characters that do not occur in the public text cannot be hidden; the tool names them. ## Structured data in a work A spreadsheet or data file can travel inside a work and be read back out of the picture. `cli/tg-data.mjs` turns
an `.xlsx`, `.csv`, `.tsv`, `.json`, `.md`, `.yaml`, `.xml`, `.ics`, `.vcf` or key-value file into **TGD 1.0**, a block of plain, whitespace-proof text (`TGD1[ ... ]TGD1`) that goes through a work unchanged, survives PDF extraction and decodes back to the data model with a checksum. ```sh node cli/tg-data.mjs encode people.xlsx -o people.tgd # the block, as text node cli/tg-data.mjs decode work.pdf --format csv # read it back out of a finished work
``` The specification, the measurements behind it and its limits are in [docs/structured-data.md](https://typographics-art.org/docs/structured-data.md). Status: library, command line, tests and the web are done. In the browser, the generator's panel *Data in this work* (`app#data`) attaches a file, shows what will be written, and either writes it into the work's text (checked by reading it back) or seals it; the work page (Machine view) shows the data as
a table with its checksum, downloads it as `.xlsx`, `.csv`, `.json` and more, and opens sealed data with a key. Everything happens in the browser, nothing is uploaded; two sample files are in `samples/data/`. Still command line only: passwords, compact book keys, a key that carries the record, and a `TGS1` block inside a work (see [docs/structured-data-ui.md](https://typographics-art.org/docs/structured-data-ui.md),
[docs/encrypted-data.md](https://typographics-art.org/docs/encrypted-data.md)). ### Sealed data Data can also be **sealed**: AES-256-GCM, with a key (`tgx1....`, 64 characters) that fits exactly one work, plus compact lossless key formats for hidden messages. `cli/tg-crypt.mjs` seals and opens; keys and passwords are passed by file, standard input or environment, never as an argument. See
[docs/encrypted-data.md](https://typographics-art.org/docs/encrypted-data.md) and [docs/key-format.md](https://typographics-art.org/docs/key-format.md). Keys are managed with one seed backup: a master key and one key per record, sealing to a recipient without knowing a secret, hidden work binding (`tgx2.`), key splitting (XOR, Shamir) and a vault (the seed alone recovers everything; a strict 2-of-3 mode exists), with Argon2id for the extra
password factor. **A password is never the only secret of a public record.** AES-256-GCM stays: see [docs/key-management.md](https://typographics-art.org/docs/key-management.md) and [docs/cipher-review.md](https://typographics-art.org/docs/cipher-review.md) (why a different cipher is not the answer). A plain-language page for non-experts explains the password rule, the three recommended setups, what is **not** promised and
how to check everything yourself: [docs/passwords-and-keys.md](https://typographics-art.org/docs/passwords-and-keys.md) (German: [docs/passwords-and-keys.de.md](https://typographics-art.org/docs/passwords-and-keys.de.md)). **Is this secure?** The cipher (AES-256-GCM, random 256-bit key) is sound; the code around it is tested but **not independently audited**, and all of the security is the secrecy of the key. You can say "cannot
be read without the key as far as today's cryptography is known", not "nobody can ever get at it": a lost key loses the data, a compromised device defeats everything, and a password alone never protects anything public. The plain answers, with the reasons, are at the top of [docs/encrypted-data.md](https://typographics-art.org/docs/encrypted-data.md). **Encrypted media and the adapter.** A picture or a video can be sealed so it only
opens with the key, and an **adapter** (`cli/tg-adapter.mjs`) makes the sealed file readable by *any* ordinary player, on the fly: it serves the plaintext at `http://127.0.0.1:...` with the right type and HTTP Range, decrypting only the frames a byte range touches, so a video seeks and streams without the plaintext ever touching the disk and without anything being uploaded. Large media is a seekable `tgst1` stream file (the one-pager is its
printable cover); a small image or a data file stays a `tgs1.` record and is served as its native format (a sealed CSV comes out as `text/csv`). Every sealed file now carries private metadata (type, name, size). See [docs/encrypted-media-adapter.md](https://typographics-art.org/docs/encrypted-media-adapter.md). An internal adversarial review of the cryptographic code by an AI agent (October 2026, with fixes and tests) is in
[docs/security-review-internal-2026-10.md](https://typographics-art.org/docs/security-review-internal-2026-10.md); a second review by a different model, done blind to the first, is in [docs/security-review-internal-2026-10-b.md](https://typographics-art.org/docs/security-review-internal-2026-10-b.md). Neither is an independent audit, and they do not replace one. ## The three rules 1. **Fidelity.** The text is never changed, reordered
or shortened. Filler text (for example Lorem ipsum for a finer picture) is allowed only if it is declared. 2. **Text stays text.** The work is published as real text: HTML, SVG, or PDF with a text layer. Extracting it yields the source text, with one space in place of each line break and run of spaces. 3. **Only typography varies.** No shapes, backgrounds or images besides the glyphs. Full definition:

# prints every setting, writes docs/file-size-study.json (about 45 minutes) node scripts/size-study.mjs --evaluate # measures the PDFs of the last run again ``` --- <!-- docs/hidden-messages.md --> ## Hidden messages # Hidden messages A Typographics Art work can carry a message that only the holder of a key can read. The work itself is not changed: it contains exactly its
public text, nothing invisible, no extra characters. The message lives only in the key. ## How it works It is a book cipher over the work's public text. 1. **The public text** is taken the way a machine reads the work: whitespace normalised, invisible format characters (soft hyphens, direction marks) removed, split into grapheme clusters (`canonicalText`
in `src/secret.js`). The same text comes out of the HTML, the SVG and the PDF (read with MuPDF). 2. **Hiding:** for every character of the secret, all its occurrences in the public text are collected and one is chosen at random (Web Crypto, without modulo bias). The key is the list of chosen positions. 3. **Fingerprint:** the key starts with the first 8 bytes of
the SHA-256 of the public text. Revealing with another work, or with a changed text, is detected and refused instead of producing garbage. 4. **Revealing:** the characters at the listed positions, joined, are the secret. ## The key `tg1.` followed by base64url of: 8 bytes text fingerprint, then the positions as variable-length integers (about 3 bytes per
character for texts of up to two million characters). The key grows by about 4 characters per character of the secret on top of a short header: measured on a 47 000-character text, 26 characters of secret gave a key of 106 characters; a 30-character secret gives about 120 to 140. ## Use ```sh node cli/tg-secret.mjs hide work.pdf --secret "Meet at the
harbour, 6 pm." # prints the key node cli/tg-secret.mjs reveal work.pdf --key tg1.... # prints the message ``` The key is the secret, so it is better not to put it on the command line, where it shows up in the process list and in the shell history: `reveal` also takes it from a file (`--key-file key.txt`), from standard input (`--key-stdin`) or from the environment
variable `TG_SECRET_KEY`. The CLI reads PDF (via MuPDF), SVG, HTML and plain text. In the [web generator](https://typographics-art.org/app), the panel *Hidden message* does the same for the current work. In code: `hideSecret(publicText, secret)` and `revealSecret(publicText, key)` from `src/index.js`. ## What it protects, and what it does not -
The work reveals nothing. Without the key it is an ordinary work; there is no hidden layer to find, because there is none. - **The key is the secret.** Whoever has the key and the work can read the message. Share the key over another channel and keep it like a password. - **Random positions:** the same secret gives a different key every time, and
repeated letters of the secret point to different places, so the key does not show which letters repeat. - **Not a replacement for encryption** where the message must stay secret even from someone who may get the key: then encrypt the message first and hide the ciphertext's characters, or use proper encryption. ## Limits - A character that does not
occur in the public text cannot be hidden; the tool names the missing characters. Longer texts contain more characters; digits and rare symbols may be absent from short ones. - The key fits exactly one text. Re-setting the same text into a new picture keeps the key valid (the text is the same); changing even one character of the text makes it invalid. -
One PDF quirk: after a character drawn from a fallback font (for example "->"), MuPDF may read a following space as missing; such a work's PDF text then differs from its source and the key made from the source does not fit the PDF (see `docs/ai-readability.md`). ## Honesty This does not touch the [rules of the
technique](https://typographics-art.org/docs/technique.md): the work contains its declared text and nothing else. Tiny or invisible text must never be used to hide anything inside a work. --- <!-- docs/passwords-and-keys.md --> ## Passwords, keys and seeds # Passwords, keys and seeds: how it works here, and what we do not promise This page is
for people who want to know whether they can trust the way Typographics Art protects data, without reading the cryptography papers. It explains the design in plain words, gives the reasons, and says plainly what is **not** covered. Everything is open source; every number below was measured or is marked as an estimate or a citation, and the last
section shows how to check it yourself. The technical documents behind it are [encrypted-data.md](https://typographics-art.org/docs/encrypted-data.md), [key-management.md](https://typographics-art.org/docs/key-management.md) and [cipher-review.md](https://typographics-art.org/docs/cipher-review.md). A German version of this page:
[passwords-and-keys.de.md](https://typographics-art.org/docs/passwords-and-keys.de.md). ## 1. In one minute - **The lock is strong.** Sealed data uses AES-256-GCM, a standard cipher that your browser or Node provides. We did not invent a cipher, and we looked at the alternatives (section 7): none is worth the risk of adding code. - **A
password is the weak point of every lock like this, so a password never stands alone.** For anything public or permanent, the real protection is a long random secret that nobody types. A password can only be an *additional* factor mixed with it. The tools refuse to publish a record that is protected by a password only. - **You hold the key; we cannot
recover it.** The page does its work in your browser and does not send your data, key, seed or password anywhere (the site loads nothing from third parties, and the browser enforces that with a content security policy). That also means: if you lose the key, the data is gone, and nobody can help you. - **It is not independently audited.** The cipher is
standard; the code around it (formats, binding, key handling) was written for this project, tested, and reviewed by automated tools, but no outside cryptographer has reviewed it. If losing or leaking the data would really hurt, get a review first. ## 2. The secrets, and what each one is for | Secret | What it is | Typed by a person? | Used for | |---|---|---|---| |
Random key (`tgx1....`, `tgx2....`) | 256 random bits, shown as about 60 characters | No: copy and paste, or a file | Opens one sealed record. This is the default protection. | | **Seed** (12, 18 or 24 recovery words, or hex) | The one backup from which a **master key** and one **key per record** are derived | Words are written down and typed rarely |
Opens everything that was sealed from it; a single record key can be given away without exposing the rest. | | **Password / passphrase** | Something a person remembers | Yes | Only as an *extra* factor (never alone for public data). | | **Share** (`tgsh1....`) | One piece of a key split in several pieces | No | A key that needs several pieces from different
hands (any k of n). | | **Passkey** (WebAuthn, the phone or a security key) | A hardware-bound secret that cannot be guessed | No: the device does it | An everyday factor of a vault (together with a passphrase; the seed stays the way back); works in a test browser with a virtual passkey, **not yet checked on real devices** (you can check yours:
`/passkey-check`). | ## 3. Why a password is the weak point A cipher with a 256-bit key is, for all practical purposes, impossible to break by guessing. A password is not a 256-bit key; it is something a human invented. What matters is how many guesses an attacker needs, and how fast they can guess. If your sealed record is **public**, anyone can
copy it and try guesses **offline, for ever**, at whatever speed their hardware allows; nobody can slow them down or notice. The table shows how long it takes to try *every* secret of a given strength. Speeds: the first column is measured on the development machine (a slow, deliberately expensive key derivation, about 10 guesses per second per
core, 32 cores, projected linearly); the other two are **estimates**, not measurements. | Strength of the secret | This machine | A GPU farm (estimate) | A state-scale effort (estimate) | |---|---|---|---| | 28 bits (a typical human password) | 9 days | 45 minutes | 27 seconds | | 40 bits (a *good* human password) | 102 years | 127 days | 1.3 days | | 50 bits | 100
000 years | 357 years | 3.6 years | | 66 bits (6 random words) | 7·10^9 years | 2.3·10^7 years | 2.3·10^5 years | | 88 bits (8 random words) | 3·10^16 years | 10^14 years | 10^12 years | | 128 bits (a random key) | 3·10^28 years | 10^26 years | 10^24 years | | 256 bits (our default) | 10^67 years | 10^65 years | 10^63 years | Two things decide the design: 1.
People do not pick random passwords. What people call a strong password is around 28 to 40 bits, and those rows fall within days to years. 2. **A human secret leaks in human ways**: typed into a page that is not ours, caught by a keylogger, written on paper, said aloud. A 256-bit key is something nobody types. (Full derivation and the exact
figures: [cipher-review.md](https://typographics-art.org/docs/cipher-review.md), section 0.1.) ## 4. The rule, and what enforces it > **A human-chosen password is never the only secret protecting a record that is public or permanent.** What the code does about it (all tested, see section 9): - The default protection of a sealed record is a random
256-bit secret, or a record key derived from your seed. - A password can be mixed in as an additional factor: the secret is then the random part and the stretched password together, so guessing the password alone gets an attacker nowhere. - `seal` with a password and no random secret is refused unless you add `--accept-password-only`, which is
meant for local files you keep to yourself. - `publish` (and the library check `assertPublishable()`) refuse a record protected by a password only. - The password factor is stretched with **Argon2id**, a memory-hard function (64 MiB and 3 passes by default, about 1 second in Chromium and 2 in Firefox): every guess costs the attacker memory and time,
which blunts GPUs. It makes guessing slower; it does not make a weak password strong. - In a **vault** (a key kept by several factors, section 5) the passphrase can never open anything alone: the code refuses to build a vault in which a set of factors that opens it contains only passphrases. ## 5. What we recommend instead (three setups) | Setup |
Who it is for | What you keep | If you lose... | |---|---|---|---| | **A. A random key, in a password manager or a file** | Most people, most data | One key per record (about 60 characters) | the key: the record is lost | | **B. One seed, many records** | Someone who seals a lot | The seed (12 to 24 words, written on paper, stored safely). Each record gets its own
key derived from it | the seed: everything sealed from it is lost; a thief with the seed opens everything | | **C. A vault: your seed, plus a passkey and a passphrase for everyday use** | Someone who wants easy access *and* a safety net | The seed backup (it alone always recovers everything, even without the vault), and for everyday use a passkey
together with a passphrase | the passkey or the passphrase: nothing, the seed still opens everything; the seed: nothing as long as you still have passkey *and* passphrase; a passphrase alone opens nothing. A thief with your seed opens everything, as in B | **The vault's default puts the seed first.** Its secret *is* the master key derived from your seed,
so the seed alone recovers everything and the vault file can be lost without harm; the price is that whoever steals the seed opens everything, exactly as in setup B. If theft of the written seed is your main risk, create the vault in strict mode (`tg-crypt vault create --mode strict`): then any two of seed, passkey and passphrase open it and the seed alone
does not, but if the vault file is lost the seed alone cannot bring the data back. In both modes a passphrase alone never opens anything; a vault that would allow it is refused. **Before you rely on a passkey, check your device** with the device check at typographics-art.org/passkey-check: it tests, in your
browser and without sending anything, whether your passkey or security key returns the same secret every time. Not every browser and device supports it, and a synced passkey may behave differently from a hardware key. Keep the seed backup as the way back in either way. **Use a separate seed for data.** Do not use the seed of a wallet that
holds money: whoever steals it would get your data as well, and a lost wallet seed would lose your data. (The platform's own wallet is watch-only and its seed is held by the operator, so it is never a place for private user data.) **A good passphrase is generated, not invented.** The tools can draw words uniformly from the BIP-39 list (2048 words, 11 bits
per word, exactly uniform): 6 words are 66 bits, 8 words 88 bits. Eight generated words are a strong *extra* factor; we still do not advise them as the *only* protection of a permanent public record (88 bits is not 128, and people handle words, not random bytes, with all the leaks of section 3). ## 6. What we do not promise - **No outside audit.** The
cipher is standard, the code around it is ours. Section 9 says what was done; none of it replaces an expert review. We recommend one before anything valuable is sealed and shared. - **The internal review is not an audit.** An AI agent reviewed the cryptographic code adversarially in October 2026
([report](https://typographics-art.org/docs/security-review-internal-2026-10.md); a second model repeated the exercise blind to the first: [report B](https://typographics-art.org/docs/security-review-internal-2026-10-b.md)): it checked the building blocks against published test vectors, wrote independent re-implementations to compare with, fuzzed
every parser, and found and fixed real defects. It is one reader with a fresh context and a shell, not a cryptographer, and it can miss what a human expert would find; do not read it as an audit. - **A lost key is lost data.** There is no recovery, no reset, no support desk that can open it. - **A compromised device defeats everything.** A malicious browser
extension, malware or a stolen unlocked phone can read what you type and see what the page decrypts. Encryption protects data at rest and in transit, not a machine that is already hostile. - **Metadata is not hidden.** A sealed record shows its size (unless padded), and whoever receives it knows that it exists and when it was shared. - **The
book-cipher hidden messages are not encryption.** They hide a message in a work's public text from someone who does not know the scheme; use sealed data for anything that must stay secret ([hidden-messages.md](https://typographics-art.org/docs/hidden-messages.md)). - **Quantum computers:** AES-256 keeps 128 bits against the best known
quantum search, which is still out of reach. The *asymmetric* part (sealing to a public key, `tgr1.`) uses X25519, which a large quantum computer would break; a hybrid with a post-quantum scheme is designed, not built, because browsers do not offer one yet. ## 7. Why we did not choose something else | Alternative | What we found | Decision | |---|---|---| | A different cipher (ChaCha20-Poly1305,
```